1. Create a new index with the updated schema definitions.
2. Copy the data from the old index into the new index using the reindexing APIs. The new index with new schema is ready to use once reindexing is complete. The index is open for both read and write operations.
3. When the new index is ready, our application switches over to it.
4. We remove the old index once we confirm that the new index works as expected.

Reindexing is a powerful operation and is discussed in detail in chapter 5, but let's take a quick look at how the API works. Suppose we wish to migrate data from an existing (`source`) index to a target (`dest`) index. We issue a reindexing call, as shown in listing 4.7.

NOTE These indices do not exist yet. This listing is only FYI.

Listing 4.7 Migrating data between indexes using reindexing API

```
POST _reindex
{
  "source": {"index": "orders"},
  "dest": {"index": "orders_new"}
}
```

The new `orders_new` index is created with the changes to the schema, and then the data from the old index (`orders`) is migrated to this newly created index with updated declarations.

NOTE If your application is rigidly coupled with an existing index, moving to the new index may require a code or configuration change. The ideal way to avoid such situations is to use aliases. *Aliases* are alternate names given to indexes. Aliasing helps us switch between indexes seamlessly with zero downtime. We look at aliases in chapter 6.

Sometimes data have incorrect types when documents are indexed. For example, a `rating` field of type `float` may receive a value enclosed as a string: `"rating": "4.9"` instead of `"rating":4.9`. Fortunately, Elasticsearch is forgiving when it encounters mismatched values for data types. It proceeds with indexing the document by extracting the value and storing it in the original data type. We discuss this mechanism in the next section.

4.3.4 Type coercion

Sometimes JSON documents contain incorrect values that differ from those in the schema definition. For example, an integer-defined field may be indexed with a string value. Elasticsearch tries to convert such inconsistent types, thus avoiding indexing problems. This process is known as *type coercion*. For example, say we've defined the `age` field of a car to be an integer type:

```
"age":{"type":"short"}
```